MICROPROJECT REPORT

AGILE & SCRUM IN DEVOPS

UNIT 2 — DEVOPS

Project Title	Applying Agile and Scrum Practices to a Student Software Project
Subject	DevOps — Unit 2: Agile & Scrum
Submitted by	Arjun Ashok Mapari
Roll Number	51
Class	AIDS (DSY)
Institute	Sanjivani College of Engineering
Academic Year	2026–2027

A practical, student-friendly study of how Agile values, Scrum roles, events, artefacts and feedback loops can support reliable software delivery in a DevOps environment.

Prepared for academic submission

1. Abstract and Introduction

Abstract. This microproject explains how Agile and Scrum help a software team plan, build, test and improve a product in short cycles. It applies these practices to a hypothetical student project: a web-based study-resource portal. The project demonstrates a product backlog, sprint planning, user stories, acceptance criteria, daily coordination, sprint review, retrospective and a simple progress-tracking method. It also explains how Agile connects with DevOps through continuous integration, testing, delivery and feedback.

Introduction

Modern software teams need to respond to changing requirements while maintaining quality and releasing improvements regularly. Traditional plans can be useful when requirements are stable, but a long delivery cycle can delay feedback. Agile encourages small increments, collaboration and learning. Scrum is a structured Agile framework that organizes work into time-boxed Sprints and defines accountabilities, events and artefacts.

Problem Statement

Student software projects can become difficult to manage when tasks are unclear, progress is not visible, testing is postponed, or team members do not communicate frequently. A lightweight and repeatable workflow is needed to break work into manageable pieces, inspect progress and improve the process.

Aim

To study and demonstrate the use of Agile and Scrum practices for managing a small software project in a DevOps-oriented workflow.

Objectives

• Explain Agile principles and compare Agile with Waterfall.
• Identify Scrum accountabilities, events and artefacts.
• Prepare a product backlog and sprint plan for a sample project.
• Define user stories and acceptance criteria.
• Show how testing, automation and feedback support DevOps.

Scope

The report is a planning and process demonstration, not a claim that a production application has been deployed. The example can be adapted to a real student project.

2. Agile Methodology and DevOps Connection

Agile is an approach to software development that emphasizes delivering useful software early and frequently, collaborating with stakeholders, adapting to change and continuously improving. Rather than treating a plan as fixed, Agile teams inspect results and adjust the next steps.

Core Agile Values

Value	Meaning in practice
Individuals and interactions	Team discussion and collaboration are prioritized over relying only on tools or formal processes.
Working software	A tested, usable increment is a stronger progress signal than a long list of unfinished documents.
Customer collaboration	Stakeholders give feedback throughout development, not only at the end.
Responding to change	The team can revise priorities when new information becomes available.

Agile vs Waterfall

Aspect	Waterfall	Agile
Planning	Most planning occurs near the start.	Planning is repeated as work evolves.
Delivery	Often delivered as a larger final release.	Delivered in smaller increments.
Feedback	Often later in the project.	Frequent feedback each iteration.
Change	Can be costly late in the process.	Handled through reprioritization.
Best fit	Stable, well-defined requirements.	Evolving needs and iterative learning.

How Agile supports DevOps

DevOps extends collaboration across development and operations. Agile helps decide what to build and in what order; DevOps practices help build, test, release and operate changes reliably. A typical loop is: plan → code → build → test → release → deploy → operate → monitor → learn. Automated tests and continuous integration make each small increment safer to integrate.

Key point: Agile and DevOps are complementary. Agile improves the flow of requirements and feedback; DevOps improves the flow and reliability of software delivery.

3. Scrum Framework: Roles, Events and Artefacts

Scrum is a lightweight framework for solving complex problems through iterative and incremental work. A Sprint is a fixed-length period of one month or less in which the team creates a valuable, usable increment. A two-week Sprint is used in this sample project.

Scrum Accountabilities

Accountability	Main responsibility	Example in the project
Product Owner	Maximizes product value and orders the Product Backlog.	Prioritizes login, search and resource-download features.
Scrum Master	Helps establish Scrum, coaches the team and addresses impediments.	Facilitates events and helps resolve blockers.
Developers	Create a usable increment and plan the work needed to meet the Sprint Goal.	Design, code, test and document portal features.

Scrum Events

Event	Purpose	Sample practice
Sprint	Container for all Scrum events and delivery work.	Two weeks, with a clear Sprint Goal.
Sprint Planning	Agree why the Sprint is valuable, what can be done and how.	Select a small set of backlog items.
Daily Scrum	Inspect progress toward the Sprint Goal and adapt the plan.	15-minute daily coordination by Developers.
Sprint Review	Inspect the outcome with stakeholders and discuss adaptations.	Demonstrate search and resource listing.
Sprint Retrospective	Identify improvements to quality and effectiveness.	Agree to test each feature before merging.

Scrum Artefacts and Commitments

- Product Backlog — ordered list of product improvements; commitment: Product Goal.
- Sprint Backlog — Sprint Goal, selected backlog items and actionable plan; commitment: Sprint Goal.
- Increment — usable, integrated work that meets the Definition of Done; commitment: Definition of Done.

4. Project Overview and Requirements

Project name: Student Study-Resource Portal (sample case study)

The proposed portal allows students to find subject-wise notes, unit resources and reference links from one place. The case study is intentionally small so that Agile and Scrum practices can be demonstrated within a classroom microproject.

Stakeholders and users

Stakeholder	Need / interest
Students	Find relevant study resources quickly and access them on mobile or desktop.
Faculty / content coordinator	Organize resources by subject and unit; keep content accurate.
Project team	Work from clear priorities, get feedback early and keep changes testable.

Functional Requirements

ID	Requirement	Priority
FR-01	Display a list of subjects and their units.	High
FR-02	Allow a student to search for a resource by keyword.	High
FR-03	Show resource title, subject, unit and resource link.	High
FR-04	Provide a simple way to submit feedback or report a broken link.	Medium
FR-05	Allow an authorized coordinator to add or update resource metadata.	Medium

Non-functional Requirements

- Usability: clear navigation and readable layouts.
- Performance: search results should appear promptly for a small dataset.
- Reliability: invalid or unavailable links should be handled gracefully.
- Security: any editing capability should require authorization.
- Maintainability: code should be organized and reviewed before integration.

Assumptions and constraints

This is a classroom case study. The sample backlog is illustrative; actual scope, team capacity, technology choices and release dates should be confirmed with the project guide and team.

5. Product Backlog and User Stories

A Product Backlog is an ordered, evolving list of work needed to improve the product. Items should be understandable, valuable, small enough to estimate and ordered by importance. The Product Owner is accountable for effective backlog management.

Sample Product Backlog

ID	User story	Priority	Estimate
US-01	As a student, I want to browse subjects so I can find my course resources.	Must	3 points
US-02	As a student, I want to search by keyword so I can locate a resource quickly.	Must	5 points
US-03	As a student, I want resource details so I can identify the correct unit and link.	Must	3 points
US-04	As a student, I want to report a broken link so the resource list stays useful.	Should	5 points
US-05	As a coordinator, I want to update resource metadata so information stays current.	Should	8 points
US-06	As a student, I want a mobile-friendly layout so I can use the portal on my phone.	Must	5 points

Example acceptance criteria — US-02: Search resources

1. Given the student is on the resource page, when a keyword is entered, then matching resource titles or subjects are shown.
2. Search ignores leading and trailing spaces.
3. If no items match, the page shows a clear “No results found” message.
4. Search works with keyboard input and remains usable on a narrow mobile screen.
5. Automated or manual checks confirm that existing resource links remain accessible.

Definition of Ready (team checklist)

Before selecting a backlog item, the team should understand the user value, have acceptance criteria, identify dependencies and agree that the item is small enough to complete within the Sprint. This checklist supports planning; it is not a replacement for conversation.

Prioritization approach

Deliver core discovery first: subject browsing, search and resource details. Add feedback reporting and coordinator editing after the basic student journey works. Revisit ordering after stakeholder feedback.

6. Sprint Planning and Execution

Sprint Goal

“Enable a student to browse subjects, search for a resource and view its key details in a simple responsive interface.”

Illustrative two-week Sprint Plan

Day / stage	Activity	Expected evidence
Day 1	Sprint Planning; confirm goal, backlog items and task plan.	Sprint Backlog and task board.
Days 2–3	Sketch page flow, define data fields and implement subject listing.	Initial interface and sample dataset.
Days 4–6	Implement keyword search and resource details; review code in small changes.	Search and details working locally.
Days 7–8	Test common and edge cases; fix defects and check mobile layout.	Test checklist and defect notes.
Day 9	Integrate changes, run regression checks and prepare demo.	Integrated increment and demo data.
Day 10	Sprint Review followed by Retrospective.	Stakeholder feedback and one improvement action.

Daily coordination

Developers inspect progress toward the Sprint Goal and adapt the plan. A useful conversation covers: what is moving toward the goal, what is next, and which impediments need attention. The Daily Scrum is for the Developers; it should not become a lengthy status report to a manager.

Sprint Backlog task board (example)

To Do	In Progress	Done
Mobile layout check	Search results component	Subject list design
Empty-state message	Resource detail view	Sample resource data

The plan is forecast-based, not a guarantee. If new work or blockers threaten the goal, Developers and Product Owner discuss scope and options while protecting the Sprint Goal.

7. Testing, Definition of Done and DevOps Workflow

Agile delivery works best when quality is built into daily work rather than postponed until the end. In a DevOps-oriented workflow, small changes are integrated frequently and verified using repeatable checks.

Definition of Done (DoD) for this sample project

A backlog item is Done when:
• Code is reviewed by another team member where practical.
• Acceptance criteria are satisfied.
• Relevant unit or component tests pass.
• The feature is integrated with the main branch without unresolved conflicts.
• Basic accessibility, responsive layout and error-state checks are completed.
• The team updates necessary documentation and demonstrates the working result.

Example test plan

Test ID	Scenario	Expected result
T-01	Open the portal with sample data.	Subject list and navigation display correctly.
T-02	Search for a word contained in a title.	Matching resource is displayed.
T-03	Search for an unmatched phrase.	Helpful empty-state message appears.
T-04	Enter a query with extra spaces.	Search still finds the expected result.
T-05	Use the page at a mobile viewport width.	Content remains readable and usable.
T-06	Open a resource whose link is unavailable.	The issue is communicated clearly; no silent failure.

Continuous Integration (CI) workflow

1. Developer creates a small branch and commits a focused change.
2. A pull request is opened for review.
3. Automated checks run: formatting/linting, unit tests and build.
4. Review comments are addressed; checks must pass before merging.
5. The merged version is tested as an integrated increment.
6. If deployment is configured, a controlled release is made and basic health checks are performed.

Important distinction

Continuous Delivery means the software is kept in a releasable state; Continuous Deployment automatically releases every qualifying change to production. A student project may use CI checks without automatically deploying to production.

8. Progress Measurement, Review and Retrospective

Progress should be evaluated using outcomes and quality, not only the number of hours spent or tasks marked complete. A Sprint Goal gives the team a shared direction. A burndown chart can show remaining estimated work over time, but it should be interpreted with discussion rather than used to blame individuals.

Illustrative Sprint Burndown Data

Sprint day	Ideal remaining points	Actual remaining points
1	24	24
2	21	22
4	17	18
6	13	12
8	8	9
10	0	2

Interpretation: two estimated points remain at the end of the example Sprint. The team should inspect why the work was not completed, clarify whether the Sprint Goal was achieved, and return unfinished items to the Product Backlog for reordering. Estimates are planning aids, not measures of personal productivity.

Sprint Review — sample questions

- Can students browse and search the resource list successfully?
- Which part of the user journey is confusing or missing?
- Are the displayed resource details sufficient?
- What should be the next highest-value improvement?

Sprint Retrospective — sample reflection

What went well?	What could improve?	Action for next Sprint
Small tasks made progress visible.	Testing started late for one component.	Write acceptance checks before implementation and test before merge.
Frequent communication surfaced a blocker.	Some backlog items were too large.	Split large items into smaller, independently testable slices.

Useful indicators

Track Sprint Goal achievement, escaped defects, build/test pass rate, time taken to restore a broken build and stakeholder feedback. Choose only metrics that help the team learn; avoid turning metrics into individual performance rankings.

9. Findings, Conclusion and References

Findings

• Breaking work into user stories makes requirements easier to discuss and validate.
• A prioritized Product Backlog helps the team focus on the highest-value student needs first.
• Scrum events create regular opportunities to plan, inspect results and adapt.
• Clear acceptance criteria and a Definition of Done reduce ambiguity and late rework.
• CI checks, code review and frequent integration support the DevOps goal of reliable delivery.
• Reviews and retrospectives serve different purposes: one inspects the product with stakeholders; the other improves the team's way of working.

Conclusion

This microproject demonstrates how Agile and Scrum can organize a small student software project into a transparent, iterative workflow. The sample Student Study-Resource Portal uses user stories, backlog prioritization, a two-week Sprint, testing criteria and a retrospective action to show how work can move from an idea toward a usable increment. Connecting this approach with DevOps practices—especially version control, automated testing, continuous integration and monitoring—helps teams shorten feedback loops and improve reliability. The framework should be adapted to the real team's context rather than followed as a rigid checklist.

Future Scope

The case study could be extended with role-based access, a content approval workflow, automated accessibility checks, a deployment pipeline, usage analytics and monitoring of broken links. Future Sprints should be selected based on student and faculty feedback.

References

1. Schwaber, K. and Sutherland, J., *The Scrum Guide*, official guide, scrumguides.org.
2. Agile Manifesto, *Manifesto for Agile Software Development*, agilemanifesto.org.
3. Atlassian, *Agile Coach: Scrum and Agile concepts*, atlassian.com/agile.
4. Google Cloud, *DevOps and DORA research resources*, cloud.google.com/devops.
5. Microsoft Learn, *Introduction to DevOps*, learn.microsoft.com.

Student Declaration

I declare that this microproject report has been prepared for academic learning. The portal and sprint data described in the report are illustrative examples unless separately implemented and verified.

Student Signature: _____ Guide Signature: _____

Date: _____ Remarks: _____